

实践8：基于 AMP 的拟人走跑策略复现



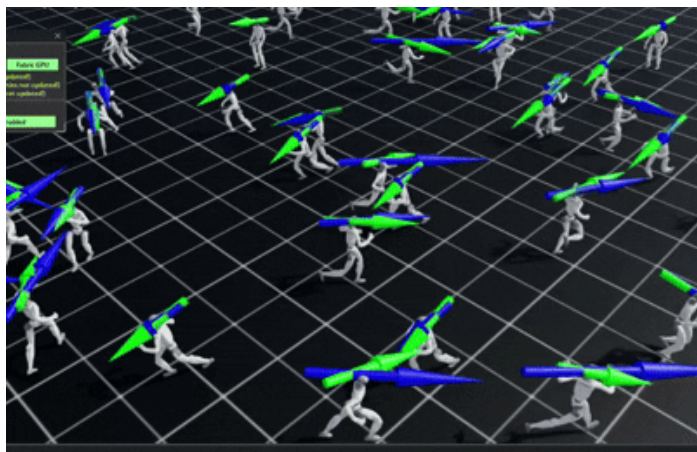
深蓝学院《人形机器人运动控制》课程第5章

课程链接：<https://www.shenlanxueyuan.com/course/864>

1. 作业目标

本项目基于 Unitree RL Lab、Isaac Lab 和 RSL-RL，使用 Adversarial Motion Priors (AMP) 训练 Unitree G1 29DoF 人形机器人完成拟人走跑。完成作业后，机器人应能够：

- 低速时行走；
- 高速时跑步；
- 在不同速度指令之间完成走跑切换；
- 根据角速度指令向左或向右转弯；
- 保持身体稳定，减少滑步等不自然姿态；
- 使策略产生的动作风格接近专家运动数据。

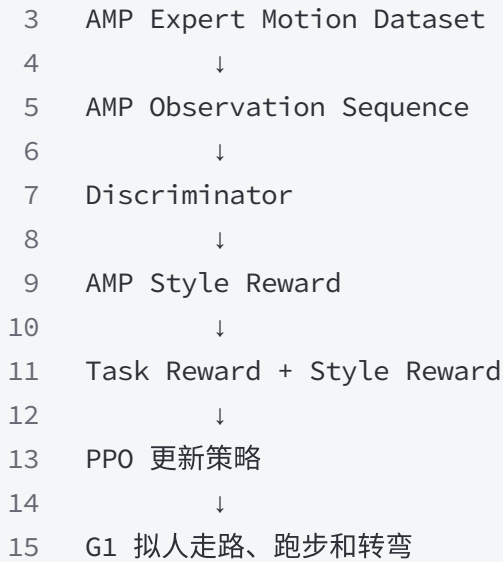


本项目不会从头实现完整强化学习框架。环境、专家数据加载、判别器训练流程和大部分 PPO 代码已经提供。你需要阅读已有实现，**补全关键 TODO（具体注释见代码）**，并完成训练、可视化和结果分析。

整体流程如下：

代码块

```
1 GMR 重定向运动数据
2 ↓
```



2. AMP 方法简介

普通速度跟踪任务主要关心机器人有没有达到目标速度，但不一定关心机器人是怎样达到该速度的。因此，仅使用任务奖励训练得到的策略可能出现动作僵硬、步态不自然、摆臂异常或高速滑步等问题。

AMP 在 PPO 训练之外增加一个判别器。判别器接收一段连续的机器人运动状态，判断该运动更接近专家数据还是策略当前产生的数据。

判别器输入不是单个时刻，而是一段长度为 H 的运动序列 $x_{\text{amp}} \in \mathbb{R}^{B \times H \times d_{\text{amp}}}$ ，其中：

- B 为并行环境数量；
- H 为 AMP 历史帧数量，本项目默认使用 3 帧；
- d_{amp} 为单帧 AMP 特征维度。

策略生成的运动窗口和专家运动窗口分别送入判别器 $D_{\psi}(x_{\text{policy}})$ 和 $D_{\psi}(x_{\text{expert}})$ ，判别器学习区分两类样本，策略则通过 AMP 风格奖励学习产生更接近专家数据的运动。

判别器网络和损失主体已经提供，但需要理解其作用。

策略样本目标接近负 margin，专家样本目标接近正 margin：

$$L_{\text{cls}} = \frac{1}{2} \mathbb{E} [(D(x_{\text{policy}}) + m)^2] + \frac{1}{2} \mathbb{E} [(D(x_{\text{expert}}) - m)^2]$$

总判别器损失还包括：

$$L_D = L_{\text{cls}} + L_{\text{logit}} + L_{\text{gp}}$$

其中梯度惩罚用于限制判别器在专家样本附近的梯度，避免判别器过快饱和。

判别器 loss 降低不代表策略一定学好了。还需要同时观察专家分数、策略分数、AMP reward 和机器人实际动作。

3. 环境安装（按照顺序执行）

3.1 获取代码

代码块

```
1 git clone https://github.com/HeYee03/unitree_lab_amp
```



备用链接，你也可以通过百度网盘获取代码：

分享文件为：unitree_lab_amp-main.zip

链接: <https://pan.baidu.com/s/1nux8Pq0QbxL3MTa8BwtYSQ?pwd=jri7>

3.2 进入根目录

代码块

```
1 cd unitree_lab_amp
```

3.3 启动环境

方案一：复用已有环境（推荐）

若原有 Unitree RL Lab 环境运行正常，可直接复用：

代码块

```
1 conda activate env_isaacrlab
```

方案二：新建独立环境（可选）

若原有环境存在兼容性问题，建议新建专属环境：

代码块

```
1 conda create -n unitree_rl_lab python=3.10 -y
2 conda activate unitree_rl_lab
```

3.4 安装项目依赖

1. 安装项目核心依赖（必做）：

代码块

```
1  ./unitree_rl_lab.sh --install
```

2. 列出已注册环境：

代码块

```
1  ./unitree_rl_lab.sh --list
```

3. 直接运行：

代码块

```
1  python scripts/list_envs.py
```

看到 Walk、Run、OmniRun 和 WalkToRun 等 AMP 任务即成功。

4. 项目文件说明

4.1 项目新增内容

本实践在原生 Unitree RL Lab 基础上扩展 AMP 相关能力，原生框架已提供基础能力如下：

- Unitree G1 机器人模型和执行器参数；
- Isaac Lab ManagerBased 环境；
- 速度指令生成；
- Actor 和 Critic 观测；
- 速度跟踪、姿态、能耗和足端相关奖励；
- PPO 训练、Play 和策略导出流程；
- G1 29DoF 平地速度任务。

本项目在此基础上增加了以下内容：

新增部分	作用
rsl_rl_amp/	在 PPO 基础上增加 AMP 算法、判别器、经验回放和 AMP Runner

amp_env.py	在 Isaac Lab 环境中接入专家动作采样器
motion_dataset.py	加载 GMR 输出的 .npz 专家动作并构造 AMP 序列
reference_reset.py	使用专家运动状态初始化部分并行环境
motion_cfg.py	管理走路、跑步、转弯和走跑切换动作数据
AMPObservationsCfg	构造仿真侧 AMP 单帧观测
AMPIsaacLabVecEnvWrapper	在环境侧维护连续 AMP 历史帧
AMPDiscriminator	区分专家运动与策略运动
ReplayBuffer	保存策略近期产生的 AMP 运动窗口
AMPRunner	检查动作 profile，并组织 PPO 与判别器联合训练
WalkToRun 任务	同时训练低速走、高速跑、走跑切换和转弯

4.2 项目目录结构

代码块

```
1  unitree_lab_amp/
2  |— rsl_rl_amp/                # AMP 强化学习算法库
3  |   |— algorithms/
4  |   |   |— ppo.py            # PPO 策略与价值网络更新
5  |   |   |— amp.py            # PPO 和 AMP 联合训练
6  |   |   |— discriminator.py  # 判别器、风格奖励和奖励混合
7  |   |   |— amp_stability.py  # 判别器训练稳定性处理
8  |   |   └ distillation.py    # 策略蒸馏
9  |   |— runners/
10 |   |   |— amp_runner.py      # AMP 训练入口
11 |   |   |— on_policy_runner.py # 通用 on-policy 训练循环
12 |   |   └ distillation_runner.py # 蒸馏训练循环
13 |   |— storage/
14 |   |   |— rollout_storage.py # PPO rollout 数据
15 |   |   └ replay_buffer.py    # 策略 AMP 样本回放
16 |   |— env/
17 |   |   |— isaac_lab_wrapper.py # Isaac Lab 环境适配和历史帧维护
18 |   |   └ vec_env.py          # 向量环境接口
19 |   |— models/                # MLP、CNN、RNN 模型封装
20 |   |— modules/              # 网络、分布和归一化模块
21 |   |— extensions/           # RND 和对称性扩展
22 |   └ utils/                  # 配置和训练日志工具
```

```

23 |
24 | └─ source/unitree_rl_lab/
25 |     └─ setup.py # Unitree Lab Python 包安装入口
26 |     └─ config/extension.toml # Isaac Lab 扩展配置
27 |     └─ unitree_rl_lab/
28 |         └─ tasks/locomotion/
29 |             └─ amp/
30 |                 └─ __init__.py
31 |                 └─ amp_env.py
32 |                 └─ motion_dataset.py
33 |                 └─ motion_sources.py
34 |                 └─ reference_reset.py
35 |                 └─ data/ # 数据，来自实践7
36 |                     └─ B1_-_stand_to_walk_stageii.npz # 数据格式样例
37 |                     └─ walk/ # 走路、倒退和走路转弯
38 |                     └─ run/ # 跑步和跑步转弯
39 |                     └─ mixed/ # 走、跑和切换动作
40 |                     └─ dance/ # 其他动作
41 |                 └─ config/g1/
42 |                     └─ __init__.py # Gym 任务注册
43 |                     └─ amp_flat_env_cfg.py
44 |                     └─ motion_cfg.py # 数据总配置文件
45 |                     └─ agents/
46 |                         └─ rsl_rl_ppo_cfg.py
47 |                 └─ robots/g1/29dof/
48 |                     └─ __init__.py
49 |                     └─ velocity_env_cfg.py
50 |                 └─ mdp/
51 |                     └─ observations.py
52 |                     └─ rewards.py
53 |                     └─ curriculums.py
54 |                     └─ commands/
55 |                         └─ velocity_command.py
56 |             └─ assets/robots/
57 |                 └─ unitree.py
58 |                 └─ unitree_actuators.py
59 |             └─ utils/
60 |                 └─ parser_cfg.py
61 |                 └─ export_deploy_cfg.py
62 |
63 | └─ scripts/
64 |     └─ list_envs.py # 显示已注册任务
65 |     └─ rsl_rl/
66 |         └─ cli_args.py # 命令行参数
67 |         └─ train.py # 训练入口
68 |         └─ play.py # Play 和模型导出入口
69 |     └─ mimic/ # 动作模仿辅助脚本

```

```

70 |
71 |— rsl_rl-main/          # 原始 RSL-RL 代码
72 |— deploy/              # ONNX 和实机部署，非本作业必做
73 |— docker/              # Docker 环境配置
74 |— unitree_rl_lab.sh    # 安装、训练和 Play 统一入口
75 |— setup.py
76 |— pyproject.toml

```

4.3 动作数据文件（实践7生成的 .npz）

`motion_cfg.py` 是本次 AMP 训练的专家运动数据总配置文件，属于环境配置层的核心文件，作用是集中管理 Unitree G1 人形机器人所有专家动作数据集的加载规则、采样权重与任务匹配关系，实现「动作数据配置」与「加载逻辑代码」的分离。数据配置位于：

代码块

```

1 source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/amp/config/g1/motion_cfg.
  py

```

动作数据建议按类型建立目录，这里的 `.npz` 文件都来自**实践7**：

代码块

```

1 amp/
2   data/
3   |— B1_-_stand_to_walk_stageii.npz
4   |— walk/
5   |   |— walk_forward.npz
6   |   |— walk_backward.npz
7   |   |— walk_turn_left.npz
8   |   |— walk_turn_right.npz
9   |— run/
10  |   |— run_forward.npz
11  |   |— run_turn_left.npz
12  |   |— run_turn_right.npz
13  |— mixed/
14  |   |— stand_to_walk.npz
15  |   |— walk_forward.npz
16  |   |— walk_to_run.npz
17  |   |— run_forward.npz
18  |   |— run_to_stand.npz
19  |   |— turn_left.npz
20  |   |— turn_right.npz

```

如果生成的文件名与 `motion_cfg.py` 中的 `clip_weights` 不一致，需要同步修改配置。

例如，配置引用了 `C5_-_walk_to_run_stageii`，但目录中没有对应文件，数据加载器会直接报缺少 motion clip。不能只把文件复制进目录而不检查配置名称。

仓库中的 `B1_-_stand_to_walk_stageii.npz` 只是格式样例，单独一条站立到行走数据不足以完成**实践 8**的低速走、高速跑和转弯训练。

4.4 关键文件说明

4.4.1 `motion_dataset.py`

该文件负责将 GMR 输出转换为 AMP 可以使用的专家运动序列，主要完成：

- 扫描指定目录中的 `.npz` 动作文件；
- 读取帧率、根节点状态、关节位置和关键连杆位置；
- 按仿真控制周期重新采样；
- 根据相邻帧计算关节速度和根节点速度；
- 将世界坐标速度转换到机器人基座坐标系；
- 根据关节名称重新排列专家动作；
- 按动作权重随机选择运动片段；
- 采样固定长度的专家运动窗口；
- 为 Reference State Initialization 提供完整专家状态。

专家数据和仿真观测必须使用相同的特征顺序。只要两侧顺序或维度不同，判别器就会比较没有对应关系的数据。

4.4.2 `motion_cfg.py`

该文件定义 G1 专家动作的数据来源与采样权重，包括：

- `walk`：前进、后退和走路转弯；
- `run`：稳定向前跑；
- `omni_run`：跑步与跑步转弯；
- `walk_to_run`：走路、跑步、走跑切换、倒退和转弯；
- `mixed`：多种运动混合；
- `dance`：舞蹈动作。

`G1WalkToRunMotionCfg` 是本作业的主要专家数据配置。不同动作的 `clip_weights` 决定训练时各片段被抽到的相对概率。

4.4.3 `amp_env.py`

该文件定义 `AMPManagerBasedRLEnv`，在普通 Isaac Lab 环境基础上增加：

- 创建专家动作采样器；
- 检查仿真 AMP 特征维度与专家特征维度是否一致；
- 保存 episode 结束前的 AMP 状态；
- 为判别器提供专家运动窗口；
- 配合参考动作重置环境。

4.4.4 `reference_reset.py`

该文件实现 Reference State Initialization。环境重置时，一部分机器人从专家动作中的随机时刻开始，而不是每次都从固定站立姿态开始。

这种方式可以让策略更快接触到走路、跑步和转弯中的不同状态。走跑任务还可以同步专家动作对应的速度指令，减少“专家状态正在跑，但命令要求慢走”这种冲突。

4.4.5 `isaac_lab_wrapper.py`

普通 Isaac Lab 环境每次只输出当前帧，而 AMP 判别器需要连续历史帧。

`AMPIsaacLabVecEnvWrapper` 负责：

- 读取名为 `amp` 的 observation group；
- 为每个并行环境维护长度为 H 的历史；
- 每一步移除最旧帧并加入最新帧；
- episode 结束时保留真正的终止帧；
- 环境重置后重新初始化历史窗口。

4.4.6 `replay_buffer.py`

该文件保存策略近期产生的 AMP 运动窗口。判别器更新时：

- 专家样本来自 `MotionDataset`；
- 策略样本来自 `ReplayBuffer`。

回放缓冲区容量固定，写满后覆盖最旧样本，使判别器既能看到近期策略，也不会只依赖当前一步的数据。

4.4.7 `discriminator.py`

该文件包含 AMP 判别器，主要负责：

- 对 AMP 单帧特征进行归一化；
- 将历史序列展平后送入 MLP；

- 计算专家样本和策略样本的分类损失；
- 计算 logit regularization；
- 计算专家样本梯度惩罚；
- 将判别器分数转换为 AMP 风格奖励；
- 混合风格奖励与任务奖励。

4.4.8 `amp.py`

`AMP` 类继承 `PPO`，并在环境采样和 `PPO` 更新之间插入 `AMP` 流程：

1. 从环境获取 `AMP` 历史序列；
2. 判断当前窗口是否已经积累足够帧数；
3. 使用判别器计算风格奖励；
4. 将有效策略窗口放入回放缓冲区；
5. 混合任务奖励和风格奖励；
6. 使用混合后的奖励计算 `return` 和 `advantage`；
7. 完成 `PPO` 更新；
8. 使用专家样本和策略样本更新判别器；
9. 记录 `AMP reward`、判别器 `loss` 和分类分数。

4.4.9 `ppo.py`

该文件实现 `PPO` 的主要训练过程，包括：

- rollout 数据收集；
- GAE `return` 和 `advantage`；
- clipped surrogate loss；
- value loss；
- policy entropy；
- 梯度裁剪和参数更新。

本作业只要求补全 `PPO` 总 `loss`，不要求重新实现完整 `PPO`。

4.4.10 `amp_flat_env_cfg.py`

该文件负责 `G1 AMP` 任务的环境配置，包括：

- `AMP observation`；
- 专家参考状态重置；
- 平地环境；

- 奖励权重；
- 速度和角速度命令范围；
- Walk、Run、OmniRun、WalkToRun 等任务；
- Train 与 Play 的环境差异；
- 完整走跑评估环境。

4.4.11 `rsl_rl_ppo_cfg.py`

该文件定义：

- Actor 和 Critic 网络规模；
- PPO 超参数；
- AMP 判别器超参数；
- 回放缓冲区大小；
- 判别器更新次数和 batch size；
- 专家动作 profile；
- 日志实验名称；
- 任务奖励与风格奖励的混合权重；
- Actor、Critic 和 AMP 的 observation group 映射。

4.4.12 `config/g1/init.py`

该文件将环境配置和 Runner 配置注册为 Gym Task ID。训练和 Play 命令中的 `--task` 参数必须与这里注册的任务名称一致。

5. 作业任务（TODO）

TODO 1：构造 AMP 单帧观测

文件路径：

`source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/amp/config/g1/amp_flat_env_cfg.py`

需要补全 `AMPObservationsCfg.AMPCfg`。

单帧 AMP 特征应包含：

特征	含义
----	----

base_lin_vel	基座坐标系线速度
base_ang_vel	基座坐标系角速度
projected_gravity	基座坐标系投影重力
base_height	基座高度
joint_pos	关节位置
joint_vel	关节速度
key_links_pos_b	双脚和双手相对基座的位置

此处的 `joint_pos` 为绝对关节角度（相对关节零位），AMP 将利用相邻帧的绝对角度差分来计算物理速度，因此数据源必须保证其为绝对设定值。

同时需要正确设置 AMP observation group：

- 不使用随机观测噪声；
- 将所有 observation term 拼接为一个向量；
- 保证特征顺序与 `motion_dataset.py` 中一致。

对于默认 G1 29DoF 和 4 个关键连杆，单帧维度应为：

$$3 + 3 + 3 + 1 + 29 + 29 + 4 \times 3 = 80$$

默认历史长度为 3，因此判别器窗口维度为 $3 \times 80 = 240$ 。

TODO 2：配置 observation group 映射

文件路径：

`source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/amp/config/g1/agents/rsl_rl_ppo_cfg.py`

需要配置 `obs_groups`，明确：

- Actor 使用哪个 observation group；
- Critic 使用哪个 observation group；
- AMP 判别器使用哪个 observation group。

完成后，Runner 应能分别读取策略观测、价值网络观测和 AMP 历史观测。

TODO 3：计算 AMP 风格奖励

文件路径：`rsl_rl_amp/algorithms/discriminator.py`

在 `style_reward()` 中，将判别器分数转换为有界的运动质量，并考虑环境步长和奖励缩放。

LSQ 模式下的运动质量为：

$$q(D_\psi(x)) = \max\left(0, 1 - \frac{1}{4}(D_\psi(x) - 1)^2\right)$$

风格奖励为：

$$r_{\text{AMP}} = \Delta t \cdot \beta \cdot q(D_\psi(x))$$

函数应返回风格奖励和原始判别器分数，便于训练日志记录。

TODO 4：混合任务奖励与风格奖励

文件路径：`rsl_rl_amp/algorithms/discriminator.py`

在 `mix_rewards()` 中实现：

$$r_t = (1 - \alpha)r_{\text{AMP}} + \alpha r_{\text{task}}$$

其中：

- r_{AMP} 约束动作风格；
- r_{task} 约束速度跟踪和稳定性；
- α 为任务奖励权重。

需要保证输入张量形状不被改变，输出能够直接交给 PPO。

TODO 5：在 AMP 环境步中计算总奖励

文件路径：`rsl_rl_amp/algorithms/amp.py`

在 `process_env_step()` 中调用判别器的奖励混合接口。

需要注意：

- 只有积累了完整历史序列的环境才能使用 AMP reward；
- 历史长度不足时应继续使用原始任务奖励；
- episode 结束后需要重新累计历史长度；
- 最终混合奖励必须传递给 PPO 的环境步处理函数。

TODO 6：计算 PPO 总 loss

文件路径：`rsl_rl_amp/algorithms/ppo.py`

需要将已经计算出的以下部分组合起来：

- clipped surrogate loss；
- value loss；
- policy entropy；
- value loss coefficient；

- entropy coefficient。

数学形式为：

$$L_{\text{PPO}} = L_{\text{policy}} + c_v L_{\text{value}} - c_e H(\pi)$$

注意熵项用于鼓励探索，因此在最小化 loss 时符号为负。

TODO 7: 补全 WalkToRun Runner 配置

文件路径：

```
source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/amp/config/g1/agents/rs1_rl_ppo_cfg.py
```

需要补全 `G1AMPWalkToRunRunnerCfg`：

- 设置独立实验名称；
- 选择走跑专家运动 profile；
- 设置任务奖励和 AMP 风格奖励的混合关系；
- 保证 Runner、环境和专家数据使用相同的 motion profile。

实验名称会决定 checkpoint 和日志保存目录，因此不能与 Walk、Run 等任务共用。

TODO 8: 构造完整走跑评估环境

文件路径：

```
source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/amp/config/g1/amp_flat_env_cfg.py
```

需要实现 `G1AMPWalkToRunFullPlayEnvCfg`。

该配置用于训练完成后的完整能力评估，应满足：

- 继承普通 WalkToRun Play 配置；
- 关闭线速度和角速度课程；
- 覆盖低速走路速度范围；
- 覆盖高速跑步速度范围；
- 支持左右转弯；
- 不启用训练阶段的观测噪声和外部扰动。

普通 Play 主要观察当前训练范围，FullPlay 用于统一验证完整速度范围。

TODO 9: 注册 FullPlay 任务

文件路径：

```
source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/amp/config/g1/__init__.py
```

需要为完整评估环境注册新的 Gym Task ID，并正确关联：

- Train environment config;
- FullPlay environment config;
- WalkToRun Runner config。

完成后，任务应出现在环境列表中，并能通过 `play.py` 启动。

6. 策略训练操作指南

完成全部 TODO 后，可执行以下命令启动训练。

6.1 基础训练

脚本方式（自动启用headless模式）：

代码块

```
1  ./unitree_rl_lab.sh --train \  
2      --task Unitree-G1-29dof-AMP-WalkToRun \  
3      --num_envs 4096
```

或者，原生Python方式：

代码块

```
1  python scripts/rsl_rl/train.py \  
2      --headless \  
3      --task Unitree-G1-29dof-AMP-WalkToRun \  
4      --num_envs 4096
```

6.2 显存适配

若显存不足，可降低并行环境数量：

代码块

```
1  ./unitree_rl_lab.sh --train \  
2      --task Unitree-G1-29dof-AMP-WalkToRun \  
3      --num_envs 1024
```

6.3 日志与模型保存

训练日志与 checkpoint 默认保存路径：

代码块

```
1 logs/rsl_rl_amp/<experiment_name>/<时间戳>/
```

补全 WalkToRun Runner 后，建议使用独立的实验名称，避免与其他动作任务的 checkpoint 混在一起。

6.4 恢复训练

代码块

```
1 ./unitree_rl_lab.sh --train \  
2     --task Unitree-G1-29dof-AMP-WalkToRun \  
3     --resume \  
4     --load_run <运行目录名称> \  
5     --checkpoint <checkpoint 文件名>
```

6.5 训练过程录屏

代码块

```
1 python scripts/rsl_rl/train.py \  
2     --headless \  
3     --task Unitree-G1-29dof-AMP-WalkToRun \  
4     --num_envs 1024 \  
5     --video \  
6     --video_length 600
```

6.6 训练监控核心指标

训练过程需重点监控以下指标，判断训练收敛性与有效性：

指标	作用
Task reward	判断速度跟踪和基础稳定性是否改善
AMP style reward	判断策略动作是否逐渐接近专家风格
Mixed reward	PPO 实际使用的总奖励
PPO surrogate loss	观察策略更新情况

Value loss	观察 Critic 拟合情况
Policy entropy	观察策略探索程度
Discriminator loss	观察判别器训练是否稳定
Expert score	判别器对专家动作的输出
Policy score	判别器对策略动作的输出
Valid window fraction	判断 AMP 历史窗口是否正确建立
Linear velocity tracking	判断低速走和高速跑是否跟随命令
Angular velocity tracking	判断左右转弯是否有效

异常排查提示：

- 如果专家分数和策略分数从训练开始就完全相同，通常说明 AMP observation、专家数据或判别器输入存在问题。
- 如果 task reward 上升但 AMP reward 长期很低，说明机器人可能完成了速度跟踪，但动作风格仍与专家数据差异较大。

7. 仿真验证与模型导出

7.1 常规回放验证

播放最新 WalkToRun checkpoint：

代码块

```
1  ./unitree_rl_lab.sh --play \  
2      --task Unitree-G1-29dof-AMP-WalkToRun \  
3      --num_envs 32
```

指定 checkpoint 回放：

代码块

```
1  ./unitree_rl_lab.sh --play \  
2      --task Unitree-G1-29dof-AMP-WalkToRun \  
3      --checkpoint /path/to/model_XXXXX.pt \  
4      --num_envs 32
```

7.2 全量能力评估

完成 FullPlay 环境和任务注册后，运行完整评估：

代码块

```
1  ./unitree_rl_lab.sh --play \  
2      --task Unitree-G1-29dof-AMP-WalkToRun-FullPlay \  
3      --checkpoint /path/to/model_xxxxx.pt \  
4      --num_envs 32
```

7.3 回放视频录制

代码块

```
1  python scripts/rs_l_rl/play.py \  
2      --task Unitree-G1-29dof-AMP-WalkToRun-FullPlay \  
3      --checkpoint /path/to/model_xxxxx.pt \  
4      --num_envs 1 \  
5      --video \  
6      --video_length 1000
```

7.4 模型导出

Play 回放过程会自动在 checkpoint 所在目录导出部署模型：

代码块

```
1  exported/policy.pt  
2  exported/policy.onnx
```

实机部署不属于本次作业的基本要求，请谨慎实机部署。

7.5 结果验收标准

训练完成后至少验证以下情况：

场景	验收内容
低速前进	能稳定行走，不明显滑步
中速前进	步频和步幅随速度增加

高速前进	能从走路过渡到跑步
减速	能从跑步回到走路或站立
左转	能跟随正向角速度指令
右转	能跟随负向角速度指令
长时间运行	不频繁摔倒或出现关节发散
有无 AMP 对比	AMP 策略的动作自然性应更接近专家数据

重点不只是“机器人有没有移动”，还应观察：

- 脚是否明显打滑；
- 身体是否持续前倾或后仰；
- 手臂是否自然摆动；
- 左右步态是否协调；
- 走跑切换是否突变；
- 转弯时是否依靠滑动完成；
- 高速状态下是否保持稳定。

8. 作业提交内容

8.1 作业提交内容

需提交以下内容完成作业：

1. 源代码：补全全部 TODO 后的完整项目代码（无需提交数据文件，请控制文件大小）
2. 训练曲线：核心监控指标的训练曲线截图
3. 效果视频：1 分钟以内的走跑效果演示视频
4. 实验报告（可选）：训练过程分析、消融实验对比、问题与解决方案等内容

8.2 提交前检查

- 所有 TODO 已补全；
- AMP 单帧观测与专家数据特征顺序一致；
- Actor、Critic 和 AMP observation group 配置正确；

- 判别器能够同时读取专家样本和策略样本；
- AMP reward、task reward 和 mixed reward 均有有效数值；
- PPO 总 loss 可以正常反向传播；
- WalkToRun 使用正确的专家动作 profile；
- FullPlay 任务能够通过命令启动；
- 策略能够低速走、高速跑并左右转弯；
- 已保存训练曲线、checkpoint、视频。

8.3 评分标准

项目	分值
专家运动数据理解与加载正确	10
AMP observation 和历史序列构造正确	20
判别器奖励及奖励混合正确	20
PPO 与 AMP 联合训练流程跑通	20
WalkToRun 环境、Runner 和任务注册	10
拟人走跑效果、对比实验与报告分析	20
总分	100

课程代码仓库：https://github.com/HeYee03/unitree_lab_amp 欢迎star!